Machine Learning : Analyse comportementale des clients

Molka Dammak - GI2 S1

2025–2026

Plan de la Présentation

1. Introduction
2. DataSet
3. Pipeline de traitement
4. Preprocessing
5. Modélisation
6. Déploiement Flask
7. Conclusion

1-INTRODUCTION

Contexte :

- L’entreprise dispose d’un grand volume de données clients issues des transactions
- Ces données contiennent des informations sur les achats et le comportement des clients

Problématique :

- Difficulté à prendre des décisions marketing efficaces à partir des données disponibles
- Manque d’outils pour identifier les clients à risque

Objectifs du Projet

Prédire le churn

Modèle de classification pour identifier clients à risque (churn = 1)

Segmenter les clients

Clustering pour profils comportementaux et tactiques marketing ciblées

Prédire dépenses futures

Modèle de régression pour estimer les dépenses futures de chaque client

2-Dataset

Description des données

- Dataset de clients d'une plateforme e-commerce
- **4372** lignes, **52** variables
- Données issues de transactions réelles

Types de variables

- **Variables numériques**

Frequency, MonetaryTotal, Age

- **Variables catégorielles**

Country, Gender, CustomerType

- **Variable cible(target)**

Churn (0 = fidèle, 1 = départ)

Problèmes détectés

- **Valeurs manquantes**

certaines variables comme **Age** contiennent beaucoup de valeurs absentes (~30%) cela peut fausser l'apprentissage du modèle

- **Valeurs aberrantes**

certaines variables ont des valeurs incohérentes

  - **Satisfaction** (valeurs -1 ou 99)
  - **SupportTickets** (valeurs extrêmes comme 999)

ces valeurs ne représentent pas la réalité

- **Données brutes non exploitables**

certaines variables ne sont pas directement utilisables

  - **IP** → besoin d'extraction (privée/publique)
  - **dates** → formats différents à convertir

3-Pipeline de traitement

Exploration des données

- comprendre la structure et détecter les problèmes

Prétraitement

- nettoyage, gestion des NaN, anomalies

Feature Engineering

- création de nouvelles variables utiles

Modélisation

- classification, clustering, régression

Évaluation

- mesurer la performance des modèles

Déploiement

- utilisation du modèle (API / prédiction)

4 -Préprocessing : Nettoyage

⚠ • Dans cette étape, nous avons préparé et transformé les données pour les rendre exploitables par les modèles de Machine Learning.

- **Suppression des colonnes inutiles :**
  - **CustomerID** → identifiant (inutile pour ML)
  - **NewsletterSubscribed** → variance nulle — toujours 'Yes'"
  - **LastLoginIP** →supprimée après extraction de IP_IsPrivate

- **Traitement des valeurs aberrantes:**
  - **SatisfactionScore :**-1, 99 → NaN
  - **SupportTickets :**999 → NaN
- **Valeurs négatives conservées :**
  - **MonetaryTotal négatif :** 44 cas conservés
- Représentent des retours clients
- Information **métier importante**

- **Parsing des dates :**

RegistrationDate →
  - **RegYear /RegMonth**
  - **RegDay / RegWeekday**

- **Transformation intelligente:**
  - **IP** Extraction d'une feature réseau **IP_IsPrivate**
  - **LastLoginIP → IP_IsPrivate**
  - **(0 = public, 1 = privé)**

5 - Preprocessing : Séparation Train/Test

Pourquoi séparer ?

- Évaluer sur des données jamais vues
- Éviter l'overfitting
- Garantir une évaluation fiable

Paramètres utilisés :

train_test_split(X, y, test_size=0.2, # 20% test random_state=42, # reproductibilité stratify=y # proportion churn identique)

Le stratify=y garantit que le taux de churn est identique dans train et test → évaluation représentative de la réalité

	Taille	% Churn
X_train	3 497 clients	33.3%
X_test	875 clients	33.3%

6- Préprocessing : Gestion des valeurs manquantes

Objectif

- Gérer les valeurs manquantes pour garantir des données complètes et exploitables

Méthode utilisée

- Médiane → 39 colonnes numériques -
- KNN Imputer → Age (30% de NaN)

Stratégie correcte

- Calcul de la médiane **uniquement sur X_train**
- Application sur : X_train et X_test
- pour préserver l'indépendance des données de test et garantir une évaluation fiable

Pourquoi la médiane ?

- Robuste aux valeurs aberrantes(outliers)
- Ne modifie pas la distribution des données
- Plus stable que la moyenne
- SatisfactionScore → NaN → médiane

Pourquoi KNN Imputer pour Age ?

- Trouve les 5 clients les plus similaires (k=5) Remplace l'âge manquant par leur moyenne
- Plus précis que la médiane pour une variable importante

7- Preprocessing : Feature Engineering & Encodage

- Création de nouvelles variables pour mieux représenter le comportement des clients

  - **AvgBasketValue**
    - calculée comme : MonetaryTotal / (Frequency + 1)
    - représente le montant moyen dépensé par transaction
    - permet d'identifier les clients à forte valeur

  - **DiversityPerTrans**
    - calculée comme : UniqueProducts / (Frequency + 1)
    - mesure la diversité des produits achetés
    - permet d'analyser le comportement d'achat
- **Pourquoi le +1 ?**

  Évite la division par zéro si Frequency = 0

- **Encodage ordinal :** AgeCategory, LoyaltyLevel, SpendingCategory
- Ces variables ont un ordre naturel et significatif — on attribue un entier croissant pour que le modèle comprenne la hiérarchie ordre logique conservé

`18-24` → 0 `25-34` → 1 `35-44` → 2 `65+` → 5

- **One-Hot Encoding :**Gender, FavoriteSeason, AccountStatus, Region, RFMSegment
- Aucune relation d'ordre entre les valeurs
- On crée une colonne binaire par catégorie.

`Gender = M` → `Gender_M = 1` `Gender_F = 0`

- **Target Encoding :** Country
- One-Hot impossible : 37+ colonnes binaires → bruit, explosion de dimension
- **Solution :** remplacer chaque pays par son taux de churn moyen, calculé uniquement sur X_train.

PAYS	CLIENTS CHURN=1	COUNTRY_ENCODED
United Kingdom	23 / 100	0.23

8-Préprocessing : Anti Data Leakage

- Qu'est-ce que le Data Leakage ? Le modèle utilise des informations qui ne seraient pas disponibles au moment de la prédiction réelle. Les résultats semblent excellents à l'entraînement, mais le modèle est inutilisable en production — il triche sans qu'on le sache.

Colonnes dérivées du churn (fuite directe)

`ChurnRiskCategory` `LoyaltyLevel` `SpendingCategory` `RFMSegment_Dormants` `RFMSegment_Fidèles` `RFMSegment_Potentiels`

Calculées à partir du comportement de churn — les garder revient à prédire le churn depuis le churn lui-même.

Colonnes postérieures à l'événement

`Recency` `CustomerTenureDays` `FirstPurchaseDaysAgo` `AccountStatus_Closed` `AccountStatus_Suspended`

Connues seulement après le départ du client — indisponibles en production au moment de prédire.

Exemple concret : ChurnRiskCategory

SANS ANTI-LEAKAGE

`ChurnRiskCategory = "Critique"`

`→ churn_pred = 1`

Accuracy 99% — mais le modèle lit directement la réponse. Inutilisable en vrai.

→

AVEC ANTI-LEAKAGE

`Frequency, MonetaryTotal, Age…`

`→ churn_pred = 1 (74% proba)`

Le modèle prédit à partir de vraies données comportementales disponibles en production.

9-Préprocessing : Normalisation & Multicolinéarité

Normalisation des données

Objectif : Mettre toutes les variables sur la même échelle

Méthode : StandardScaler

Principe :

- moyenne = 0
- variance = 1

Application :

- Fit sur X_train
- Transform sur X_train et X_test

Intérêt :

- Améliore les modèles basés sur distance (KMeans, PCA)
- Stabilise l'apprentissage

Multicolinéarité (VIF)

Définition : Corrélation forte entre variables

Problèmes :

- Redondance d'information
- Instabilité du modèle

Mesure : VIF (Variance Inflation Factor)

Interprétation :

- VIF < 5 → acceptable
- VIF > 10 → problématique

Action : ariables conservées — gérées par l'anti-leakage et la PCA

Résultat :

- Modèle plus robuste
- Meilleure interprétation

9 -Modélisation :Classification

Pourquoi Random Forest ?

- Ensemble de plusieurs arbres de décision chaque arbre vote et la majorité gagne. Plus robuste qu'un seul arbre.
- Robuste au surapprentissage sur données mixtes
- class_weight='balanced' : compense le déséquilibre churn (classe rare ~30%)
- Donne une importance par feature — on sait ce qui explique le churn

Optimisation par Optuna

- Framework d'optimisation bayésienne — explore intelligemment l'espace des hyperparamètres en apprenant des essais précédents.
- Plus intelligent que GridSearch : explore l'espace intelligemment
- Paramètres optimisés : n_estimators, max_depth, min_samples_split, max_features

81.1%	0.856	0.68
Accuracy	AUC-ROC	F1 CV-5 ± 0.016

10 - Modélisation : Clustering (PCA + KMeans)

Étape 1 — Réduction PCA (19 composantes)

- Pourquoi PCA avant KMeans ? En haute dimension les distances perdent leur sens
- 90.1% de variance expliquée avec 19 composantes
- 262 outliers retirés (IQR × 1.5) avant le fit pour un clustering plus propre

Étape 2 — KMeans k=3

- Méthode du coude — k testé de 2 à 8, k=3 retenu
- n_init=10 — 10 initialisations pour éviter les minima locaux
- Algorithme non supervisé — ne voit pas le churn
- 3 497 clients d'entraînement → 3 segments distincts

3 497
clients clusterisés

90.1%
variance expliquée (PCA 19)

262
outliers retirés avant fit

Profils des 3 segments — valeurs normalisées, comparaison relative (PCA 19 composantes)

	Churn rate	Frequency	Monetary	AvgDays
Cluster 0 — Meilleurs clients 771 clients · 22.0% du total Réactivation prioritaire	25.4%	+0.022	+0.040	-0.224
Cluster 1 — Actifs standard 1599 clients · 45.7% du total Fidélisation ciblée	35.0%	+0.054	+0.055	+0.083
Cluster 2 — Faible valeur 1127 clients · 32.2% du total Rétention urgente	36.1%	-0.092	-0.105	+0.035

11 -Régression (Dépenses futures)

Prédiction MonetaryTotal

- **Objectif :** Estimer la dépense future de chaque client (MonetaryTotal)
- **Modèle principal : XGBoost Regressor**

Pourquoi XGBoost ?

- **Très performant sur données tabulaires**
- **Capable de modéliser des relations non linéaires**
- **Gestion robuste des outliers**
- **Meilleure précision que Random Forest dans notre cas**

Améliorations appliquées

- Transformation log1p sur la cible → stabilise les variations
- Modélisation en 2 régimes (small / large clients)
- Clipping pour limiter les valeurs extrêmes
- Feature engineering avancé

MAE

359.95 £ (26.7%)

On estime la dépense à ± 359£ en moyenne (26.7%)

RMSE

779.67 £ (57.8%)

pénalise les gros écarts
Sensible aux clients à très forte dépense (57.8%)

R^2

0.8248

variance expliquée
82% de la variance du CA expliquée

12-Déploiement Flask

API REST

- **GET/ :** Interface HTML — formulaire client
- **GET/random_client :**

Pioche un client depuis data.csv → pré-remplit le formulaire automatiquement

- **POST/predict :**

JSON client → churn_pred, churn_proba, risk_level, cluster, monetary_pred

Pipeline d'inférence — /predict

1. JSON reçu → transform_for_inference()

même preprocessing qu'à l'entraînement

2. scaler.transform() → normalisation StandardScaler
3. model.predict() → churn + probabilité + risk_level
4. pca_cluster + kmeans → segment client
5. scaler_reg + reg_model → dépense estimée £
6. JSON renvoyé avec les 3 résultats

13-Bilan du Projet: Conclusion & Perspectives

Points Forts

Pipeline complet de Machine Learning
- Préprocessing → Modélisation → Déploiement

Approche multi-modèles
- Classification (churn)
- Clustering (segmentation)
- Régression (dépenses futures)

Résultats satisfaisants
- Bonne précision (≈ 80.57%)
- Modèle stable et cohérent

Projet concret
- Application réelle avec Flask
- Prédictions en temps réel

Perspectives

Améliorer la qualité des données
- Ajouter plus de données pour enrichir le dataset

Amélioration des modèlesOptimisation avancée (stacking, ensemble)
- Tests de modèles plus complexes

Industrialisation
- Déploiement cloud (API scalable)
- Monitoring des performances en production

Merci pour votre attention